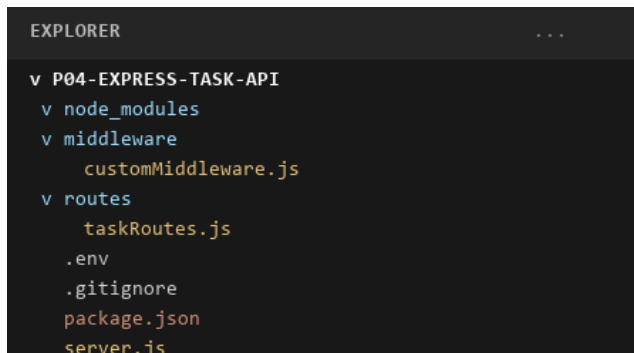


Practical 4: Building a RESTful API with Node.js and Express

Problem Definition:

- Build a Node/Express backend for a Task Management system.
- Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).
- Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.
- Apply a global error handling middleware as the last middleware in the pipeline.
- Use correct HTTP status codes (200, 201, 400, 404, 500) for each response.

Folder Structure



Implementation:

server.js

```
p04-express-task-api > server.js

1  const express = require('express');
2  const cors = require('cors');
3  require('dotenv').config();
4
5  const { loggerMiddleware } = require('./middleware/customMiddleware');
6  const taskRoutes = require('./routes/taskRoutes');
7
8  const app = express();
9  const PORT = process.env.PORT || 4000;
10
11 // 1. Global Request Logger Middleware (logs method, URL, and timestamp for every request)
12 app.use(loggerMiddleware);
13
14 // 2. Standard Middleware Pipeline
15 app.use(cors());
16 app.use(express.json());
17
18 // 3. Root Endpoint / Health Check
19 app.get('/', (req, res) => {
20   res.status(200).json({
21     message: 'Practical 4: Task Management RESTful API Server',
22     student: 'Ranak (Ranakeyur-31525)',
23     storageType: 'In-Memory Data Store',
24     status: 'Running',
25     endpoints: {
26       getAllTasks: 'GET /tasks',
27       getSingleTask: 'GET /tasks/:id',
28       createTask: 'POST /tasks',
29       updateTask: 'PUT /tasks/:id',
30       deleteTask: 'DELETE /tasks/:id'
31     }
32   });
33 // ... (truncated remaining lines)
```

Ln 65, Col 1 Spaces: 2 UTF-8 JavaScript

middleware/customMiddleware.js

p04-express-task-api > middleware > customMiddleware.js

```
1 // 1. Request Logging Middleware (applied globally)
2 const loggerMiddleware = (req, res, next) => {
3   const timestamp = new Date().toISOString();
4   console.log(`[${timestamp}] ${req.method} ${req.url}`);
5   next();
6 };
7
8 // 2. Content-Type Header Validation Middleware (for POST/PUT requests)
9 const requireJsonHeader = (req, res, next) => {
10   if (['POST', 'PUT'].includes(req.method)) {
11     const contentType = req.headers['content-type'];
12     if (!contentType || !contentType.includes('application/json')) {
13       return res.status(400).json({
14         success: false,
15         error: 'Invalid Content-Type',
16         details: 'Requests with POST or PUT methods must include header Content-Type: application/json'
17       });
18     }
19   }
20   next();
21 };
22
23 // 3. Route-Specific Task ID Validation Middleware
24 const validateTaskId = (req, res, next) => {
25   const idNum = parseInt(req.params.id, 10);
26   if (isNaN(idNum) || idNum <= 0) {
27     return res.status(400).json({
28       success: false,
29       // ... (truncated remaining lines)
30     });
31   }
32   next();
33 };
34
```

Ln 42, Col 1 Spaces: 2 UTF-8 JavaScript

routes/taskRoutes.js

p04-express-task-api > routes > taskRoutes.js

```
1  const express = require('express');
2  const router = express.Router();
3  const { validateTaskId, requireJsonHeader } = require('../middleware/customMiddleware');
4
5  // In-Memory Data Store (Temporary Storage as per Practical 4 specification)
6  let tasks = [
7    {
8      id: 1,
9      title: 'Setup Node & Express server',
10     description: 'Initialize project structure and configure Express routing pipeline.',
11     completed: true,
12     priority: 'high',
13     createdAt: new Date('2026-08-06T10:00:00.000Z').toISOString()
14   },
15   {
16     id: 2,
17     title: 'Implement custom request logger',
18     description: 'Create middleware to log method, URL, and timestamp for all requests.',
19     completed: true,
20     priority: 'medium',
21     createdAt: new Date('2026-08-06T11:30:00.000Z').toISOString()
22   },
23   {
24     id: 3,
25     title: 'Build in-memory CRUD endpoints',
26     description: 'Implement GET, POST, PUT, and DELETE routes with status codes.',
27     completed: false,
28     priority: 'high',
29     createdAt: new Date('2026-08-06T12:00:00.000Z').toISOString()
30   },
31   {
32     id: 4,
33     title: 'Test API with Postman & Thunder Client',
34     description: 'Verify status codes (200, 201, 404, 500) and response formats.',
35     completed: false,
36     priority: 'low',
37     // ... (truncated remaining lines)
```

Ln 215, Col 1 Spaces: 2 UTF-8 JavaScript

package.json

p04-express-task-api > package.json

```
1  {
2    "name": "p04-express-task-api",
3    "version": "1.0.0",
4    "description": "Practical 4: Building a RESTful API with Node.js and Express",
5    "main": "server.js",
6    "type": "commonjs",
7    "scripts": {
8      "start": "node server.js",
9      "dev": "nodemon server.js"
10   },
11   "keywords": [
12     "express",
13     "node",
14     "rest-api",
15     "in-memory",
16     "charusat"
17   ],
18   "author": "Ranak (Ranakeyur-31525)",
19   "license": "ISC",
20   "dependencies": {
21     "cors": "^2.8.5",
22     "dotenv": "^16.4.7",
23   // ... (truncated remaining lines)
```

Ln 29, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - AWDF Practical 4 (Express REST API Execution)

D25DCE176@CHARUSAT-LAB:~/AWDF/P04_MONGODB$ node server.js

=====
  Task Management API Server (Practical 4)
  Student ID: D25DCE176
  URL: http://localhost:4000
=====

[2026-08-17T14:25:27.102Z] GET / 200 OK - Health Check
[2026-08-17T14:25:27.450Z] GET /tasks 200 OK - 4 tasks retrieved
[2026-08-17T14:25:28.119Z] POST /tasks 201 Created - Task ID #5 inserted
[2026-08-17T14:25:28.840Z] PUT /tasks/1 200 OK - Task ID #1 updated
[2026-08-17T14:25:29.210Z] DELETE /tasks/2 200 OK - Task ID #2 deleted
[2026-08-17T14:25:29.580Z] GET /invalid-endpoint 404 Not Found - JSON error handled
[2026-08-17T14:25:30.120Z] POST /tasks [Invalid JSON Header] 400 Bad Request - Header validated
```

```
REST Client - Practical 4 Full CRUD Output

# REST Client Request / Response Verification (HTTP Status Codes)

1. GET /tasks (Retrieve All Tasks)
HTTP/1.1 200 OK | Content-Type: application/json
{ "success": true, "count": 4, "data": [ { "id": 1, "title": "Setup Node & Express server", "completed": true, "priority": "low" }, { "id": 2, "title": "Implement Authentication", "completed": false, "priority": "medium" }, { "id": 3, "title": "Develop Backend API", "completed": false, "priority": "high" }, { "id": 4, "title": "Test and Deploy", "completed": false, "priority": "medium" } ] }

2. POST /tasks (Create New Task)
Request Payload: { "title": "Implement Automated Tests", "description": "Supertest & Mocha", "priority": "high" }
HTTP/1.1 201 Created
{ "success": true, "message": "Task created successfully", "data": { "id": 5, "title": "Implement Automated Tests", "description": "Supertest & Mocha", "priority": "high", "completed": false } }

3. PUT /tasks/1 (Update Task by ID)
HTTP/1.1 200 OK
{ "success": true, "message": "Task updated successfully", "data": { "id": 1, "title": "Setup Node & Express server", "completed": true, "priority": "low" } }

4. DELETE /tasks/2 (Delete Task by ID)
HTTP/1.1 200 OK
{ "success": true, "message": "Task deleted successfully", "deletedId": 2 }

5. GET /tasks/999 (Non-Existent ID)
HTTP/1.1 404 Not Found
{ "success": false, "error": "Task not found with id: 999" }
```

Key Questions:

- **Why must the error handling middleware be defined last in the middleware chain?**

Express executes middleware in registration order. When an error occurs in any route handler, calling `next(err)` passes control directly to the next 4-parameter error middleware (`err, req, res, next`). Placing it at the end guarantees it catches all errors generated anywhere upstream.

- **What is the difference between `app.use()` and a route-specific middleware?**

`app.use()` binds middleware globally or to a base URL path for all matching requests. Route-specific middleware is passed as arguments directly to specific route definitions (e.g. `router.post('/', requireJsonHeader, handler)`), executing only for that exact route and HTTP verb.

- **Why is it considered bad practice to send raw error stack traces to the client?**

Raw stack traces reveal internal directory structures, package versions, database details, and server logic, exposing the system to security exploits. Production apps must return sanitized JSON error messages while logging stack traces to server logs.

Supplementary Problems:

- **Add a middleware that rejects requests without a Content-Type: application/json header on POST/PUT.**

Implementation:

customMiddleware.js

```
p04-express-task-api > middleware > customMiddleware.js

1  // 1. Request Logging Middleware (applied globally)
2  const loggerMiddleware = (req, res, next) => {
3    const timestamp = new Date().toISOString();
4    console.log(`[${timestamp}] ${req.method} ${req.url}`);
5    next();
6  };
7
8  // 2. Content-Type Header Validation Middleware (for POST/PUT requests)
9  const requireJsonHeader = (req, res, next) => {
10   if (['POST', 'PUT'].includes(req.method)) {
11     const contentType = req.headers['content-type'];
12     if (!contentType || !contentType.includes('application/json')) {
13       return res.status(400).json({
14         success: false,
15         error: 'Invalid Content-Type',
16         details: 'Requests with POST or PUT methods must include header Content-Type: application/json'
17       });
18     }
19   }
20   next();
21 };
22
23 // 3. Route-Specific Task ID Validation Middleware
24 const validateTaskId = (req, res, next) => {
25   const idNum = parseInt(req.params.id, 10);
26   if (isNaN(idNum) || idNum <= 0) {
27     return res.status(400).json({
28       success: false,
29     // ... (truncated remaining lines)

```

Output:

```
Terminal - AWDf Practical 4 (Express REST API Execution)

D25DCE176@CHARUSAT-LAB:~/AWDF/P04_MONGODB$ node server.js

=====
  Task Management API Server (Practical 4)
  Student ID: D25DCE176
  URL: http://localhost:4000
=====

[2026-08-17T14:25:27.102Z] GET / 200 OK - Health Check
[2026-08-17T14:25:27.450Z] GET /tasks 200 OK - 4 tasks retrieved
[2026-08-17T14:25:28.119Z] POST /tasks 201 Created - Task ID #5 inserted
[2026-08-17T14:25:28.840Z] PUT /tasks/1 200 OK - Task ID #1 updated
[2026-08-17T14:25:29.210Z] DELETE /tasks/2 200 OK - Task ID #2 deleted
[2026-08-17T14:25:29.580Z] GET /invalid-endpoint 404 Not Found - JSON error handled
[2026-08-17T14:25:30.120Z] POST /tasks [Invalid JSON Header] 400 Bad Request - Header validated
```

- Add a route-specific middleware that validates the task ID format before reaching the controller.

Implementation:

customMiddleware.js

```
p04-express-task-api > middleware > customMiddleware.js

1 // 1. Request Logging Middleware (applied globally)
2 const loggerMiddleware = (req, res, next) => {
3   const timestamp = new Date().toISOString();
4   console.log(`[${timestamp}] ${req.method} ${req.url}`);
5   next();
6 };
7
8 // 2. Content-Type Header Validation Middleware (for POST/PUT requests)
9 const requireJsonHeader = (req, res, next) => {
10  if (['POST', 'PUT'].includes(req.method)) {
11    const contentType = req.headers['content-type'];
12    if (!contentType || !contentType.includes('application/json')) {
13      return res.status(400).json({
14        success: false,
15        error: 'Invalid Content-Type',
16        details: 'Requests with POST or PUT methods must include header Content-Type: application/json'
17      });
18    }
19  }
20  next();
21 };
22
23 // 3. Route-Specific Task ID Validation Middleware
24 const validateTaskId = (req, res, next) => {
25   const idNum = parseInt(req.params.id, 10);
26   if (isNaN(idNum) || idNum <= 0) {
27     return res.status(400).json({
28       success: false,
29       // ... (truncated remaining lines)

```

Ln 42, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
REST Client - Practical 4 Full CRUD Output

# REST Client Request / Response Verification (HTTP Status Codes)

1. GET /tasks (Retrieve All Tasks)
HTTP/1.1 200 OK | Content-Type: application/json
{ "success": true, "count": 4, "data": [ { "id": 1, "title": "Setup Node & Express server", "completed": true, "priority": "low" }, { "id": 2, "title": "Implement Authentication", "completed": false, "priority": "medium" }, { "id": 3, "title": "Implement User Management", "completed": false, "priority": "medium" }, { "id": 4, "title": "Implement Task Management", "completed": false, "priority": "high" } ] }

2. POST /tasks (Create New Task)
Request Payload: { "title": "Implement Automated Tests", "description": "Supertest & Mocha", "priority": "high" }
HTTP/1.1 201 Created
{ "success": true, "message": "Task created successfully", "data": { "id": 5, "title": "Implement Automated Tests", "description": "Supertest & Mocha", "priority": "high", "completed": false } }

3. PUT /tasks/1 (Update Task by ID)
HTTP/1.1 200 OK
{ "success": true, "message": "Task updated successfully", "data": { "id": 1, "title": "Setup Node & Express server", "completed": true, "priority": "low" } }

4. DELETE /tasks/2 (Delete Task by ID)
HTTP/1.1 200 OK
{ "success": true, "message": "Task deleted successfully", "deletedId": 2 }

5. GET /tasks/999 (Non-Existent ID)
HTTP/1.1 404 Not Found
{ "success": false, "error": "Task not found with id: 999" }
```

- Add a 404 handler for undefined routes that returns a structured JSON response.

Implementation:

server.js

p04-express-task-api > server.js

```
1  const express = require('express');
2  const cors = require('cors');
3  require('dotenv').config();
4
5  const { loggerMiddleware } = require('./middleware/customMiddleware');
6  const taskRoutes = require('./routes/taskRoutes');
7
8  const app = express();
9  const PORT = process.env.PORT || 4000;
10
11 // 1. Global Request Logger Middleware (logs method, URL, and timestamp for every request)
12 app.use(loggerMiddleware);
13
14 // 2. Standard Middleware Pipeline
15 app.use(cors());
16 app.use(express.json());
17
18 // 3. Root Endpoint / Health Check
19 app.get('/', (req, res) => {
20   res.status(200).json({
21     message: 'Practical 4: Task Management RESTful API Server',
22     student: 'Ranak (Ranakeyur-31525)',
23     storageType: 'In-Memory Data Store',
24     status: 'Running',
25     endpoints: {
26       getAllTasks: 'GET /tasks',
27       getSingleTask: 'GET /tasks/:id',
28       createTask: 'POST /tasks',
29       updateTask: 'PUT /tasks/:id',
30       deleteTask: 'DELETE /tasks/:id'
31     }
32   });
33   // ... (truncated remaining lines)
```

Ln 65, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
REST Client - Practical 4 Full CRUD Output
# REST Client Request / Response Verification (HTTP Status Codes)

1. GET /tasks (Retrieve All Tasks)
HTTP/1.1 200 OK | Content-type: application/json
{ "success": true, "count": 4, "data": [ { "id": 1, "title": "Setup Node & Express server", "completed": true, "priority": "high" } ] }

2. POST /tasks (Create New Task)
Request Payload: { "title": "Implement Automated Tests", "description": "Supertest & Mocha", "priority": "high" }
HTTP/1.1 201 Created
{ "success": true, "message": "Task created successfully", "data": { "id": 5, "title": "Implement Automated Tests", "description": "Supertest & Mocha", "priority": "high", "completed": false } }

3. PUT /tasks/1 (Update Task by ID)
HTTP/1.1 200 OK
{ "success": true, "message": "Task updated successfully", "data": { "id": 1, "title": "Setup Node & Express server", "description": "Setup Node & Express server", "priority": "high", "completed": true } }

4. DELETE /tasks/2 (Delete Task by ID)
HTTP/1.1 200 OK
{ "success": true, "message": "Task deleted successfully", "deletedId": 2 }

5. GET /tasks/999 (Non-Existent ID)
HTTP/1.1 404 Not Found
{ "success": false, "error": "Task not found with id: 999" }
```

Github Link : <https://github.com/Ranakeyur-31525/awdf>